AWS MGN | REHOST TO EC2 | REPLATFORM TO RDS

SQL Server Migration with AWS MGN

On-premises to EC2, then EC2 to RDS

Architecture, planning, step-by-step runbooks, cutover and troubleshooting for DBAs using AWS Transform MGN (formerly Application Migration Service)



WHAT YOU WILL LEARN

- Where MGN fits (and where it does not)
- MGN architecture, ports, lifecycle and quotas
- Planning: bandwidth, licensing, AD, SQL Server specifics
- Hands-on runbook: install, replicate, test, cut over
- Second leg: EC2 to Amazon RDS with native backup or DMS

Prepared October 2026. Versions, limits and features change often; verify exact values in the AWS documentation.

Contents

How to Use This Guide . . . 4

1. Where AWS MGN Fits in a SQL Server Migration . . . 5

1.1 The two-leg journey . . . 5

1.2 What MGN is . . . 5

1.3 What MGN does not do . . . 6

1.4 Choosing the right tool . . . 6

2. MGN Architecture and How It Works . . . 7

2.1 Components . . . 7

2.2 Network ports and endpoints . . . 8

2.3 Source-server lifecycle . . . 8

2.4 Data replication states . . . 9

2.5 Quotas to know . . . 9

2.6 Cost model . . . 9

3. Planning a SQL Server Migration with MGN . . . 10

3.1 Assessment checklist . . . 10

3.2 Network and bandwidth . . . 11

3.3 AWS landing zone . . . 11

3.4 Licensing decisions . . . 11

3.5 Active Directory, DNS and names . . . 12

3.6 SQL Server specific preparation . . . 12

3.7 Sizing the EC2 target . . . 12

4. Hands-On: On-Premises SQL Server to EC2 with MGN . . . 13

4.1 Step 1: Prepare AWS . . . 13

4.2 Step 2: Create credentials for the agent . . . 14

4.3 Step 3: Install the AWS Replication Agent . . . 14

4.4 Step 4: Monitor replication . . . 15

4.5 Step 5: Configure launch settings for the SQL Server . . . 15

4.6 Step 6: Launch a test instance and validate . . . 15

4.7 Step 7: Plan the cutover . . . 16

4.8 Step 8: Cutover day . . . 17

4.9 Step 9: Immediately after launch (SQL Server fixes) . . . 18

5. After the Move: Optimising SQL Server on EC2 . . . 19

5.1 First-week checklist . . . 19

5.2 Storage and performance tuning . . . 19

5.3 High availability and disaster recovery on EC2 . . . 20
5.4 Clean up and cost control . . . 20
5.5 Troubleshooting MGN for SQL Server . . . 21
6. Second Leg: SQL Server on EC2 to Amazon RDS . . . 22
6.1 Why MGN is not used for EC2 to RDS . . . 22
6.2 Assess RDS readiness . . . 23
6.3 Choosing the method . . . 23
6.4 Prepare the RDS target . . . 24
6.5 Path A: native backup and log restore . . . 24
6.6 Path B: AWS DMS from EC2 to RDS . . . 25
6.7 Server-level objects . . . 25
6.8 Validation, cutover and clean-up . . . 26
6.9 A combined programme plan . . . 26
Appendix A: MGN Command Cheat Sheet . . . 27
Appendix B: Cutover Readiness Checklist . . . 27
Appendix C: Glossary . . . 27
Appendix D: Where to Learn More . . . 28

How to Use This Guide

This guide explains how to use AWS MGN (AWS Transform MGN, formerly AWS Application Migration Service) to migrate SQL Server from on-premises to Amazon EC2, and how to continue from EC2 to Amazon RDS. It is written for DBAs and includes diagrams, runbooks, commands, checklists and troubleshooting tables.

Chapter	You will learn	Read it when you...
1. Where MGN Fits	What MGN does and does not do; the two-leg journey; choosing tools	must pick a strategy
2. MGN Architecture	Agent, replication servers, staging, ports, lifecycle, quotas, cost	design or troubleshoot
3. Planning	Assessment, bandwidth, landing zone, licensing, AD, SQL specifics, sizing	prepare a wave
4. Hands-On Runbook	Install the agent, replicate, test, clean cutover, post-launch fixes	run the migration
5. After the Move	Optimise, protect, patch, monitor, clean up, troubleshoot	operate on EC2
6. EC2 to RDS	Why MGN cannot do it; native backup/log restore, DMS, cutover, plan	modernise to RDS

The key point

MGN moves **servers** to **EC2**. It cannot place a database into **Amazon RDS**. For EC2 to RDS you use database-level methods (native backup and restore, AWS DMS or replication). This guide covers both legs.

A note on accuracy

AWS renamed Application Migration Service to AWS Transform MGN in June 2026. Quotas, supported operating systems, pricing terms, CLI options and RDS version support change often. Values here are planning guidance; confirm in the current AWS documentation before you build or cut over.

1. Where AWS MGN Fits in a SQL Server Migration

AWS MGN is the AWS service for **lift-and-shift** (rehost) migrations. It copies a whole server, disk by disk, to Amazon EC2 and starts it there. This guide shows how to use it to move SQL Server from on-premises to EC2, and, because MGN cannot place a database into RDS, how to take the second step from EC2 to Amazon RDS with the right tools.

Important: what MGN can and cannot do

Can: migrate physical, VMware, Hyper-V and other servers (including a Windows server running SQL Server) to **Amazon EC2** with minimal downtime, keeping the OS, SQL Server version, databases, jobs and settings exactly as they are.
Cannot: migrate a database **into Amazon RDS**. RDS has no operating system for MGN to replicate. EC2 to RDS needs database-level tools: native backup/restore, AWS DMS or replication (chapter 6).
Naming: in June 2026 AWS renamed Application Migration Service to **AWS Transform MGN**. It is the same replication engine and console. This guide says 'MGN'.

1.1 The two-leg journey

Many teams use a **two-step path**: first rehost quickly with MGN to leave the data centre with low risk, then modernise databases to RDS at their own pace.

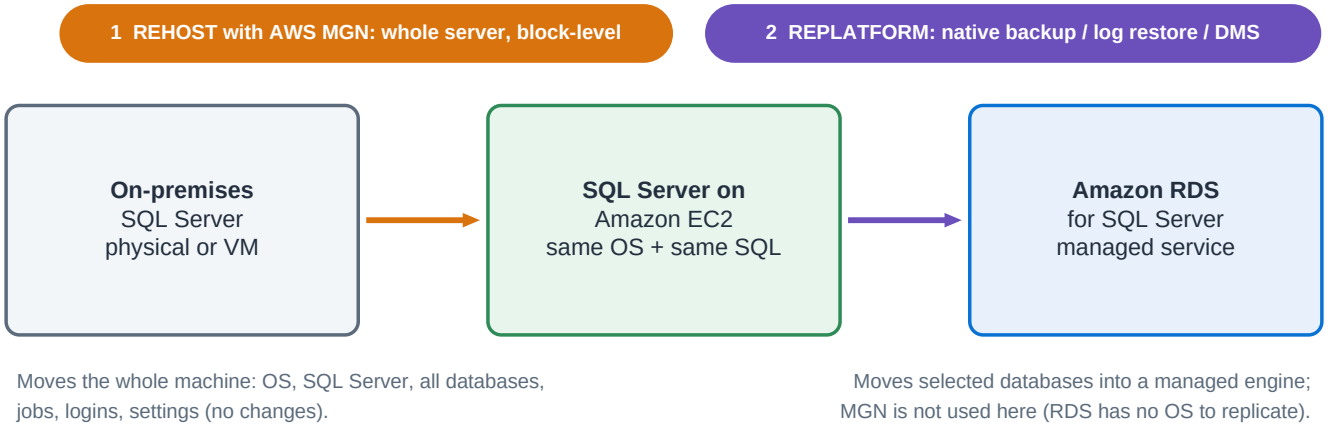


Figure 1.1 - MGN rehosts to EC2; a second step replatforms to RDS

1.2 What MGN is

MGN installs a small **AWS Replication Agent** on each source server. The agent reads every disk block once (initial sync) and then continuously sends every change to a cheap staging area in your AWS account. When you are ready, MGN converts the replicated disks into a bootable EC2 instance (fixing drivers and boot settings) and launches it. You can launch **test** instances as often as you like while replication continues, then do the final **cutover**.

Feature	What it gives a SQL Server DBA
Continuous block-level replication	Near-zero RPO: the copy follows the source all the time, so the final switch needs only a short outage

Feature	What it gives a SQL Server DBA
Non-disruptive testing	Launch a full copy of the production server in AWS for validation without stopping replication or touching the source
Automated conversion	Drivers and boot configuration are adapted for EC2; no manual P2V work
Whole-server fidelity	All databases, system databases, SQL Agent jobs, logins, linked servers, certificates and OS settings come along
Post-launch actions	Automatically install the SSM agent, CloudWatch agent, convert SQL Server BYOL to licence-included, run custom scripts
Wave and application grouping	Group servers (for example SQL + app servers) and cut them over together

1.3 What MGN does not do

- **No change of SQL Server version or edition.** The EC2 copy is the same Windows and the same SQL Server as the source (apart from optional OS upgrade actions). Upgrades and modernisation come afterwards.
- **Not database-aware.** It copies disk blocks. It does not choose databases, shrink files, change collation or read the transaction log.
- **Crash-consistent copies.** Without quiescing, the replica is like the disk after a sudden power loss. SQL Server recovers from this normally at startup, but a clean shutdown before cutover is best (section 4.9).
- **Cannot target RDS or other managed databases.**
- **Clusters and shared storage need special treatment.** Failover Cluster Instances with shared disks are generally rebuilt on AWS rather than replicated as-is.

1.4 Choosing the right tool

Goal	Best tool	Why
Move the entire SQL Server machine unchanged, fast	**AWS MGN**	Whole-server, low risk, minimal downtime, test as often as needed
Move to a newer SQL Server version while migrating	Native backup/restore, log shipping or DMS to a new EC2/RDS	MGN keeps the old version
Move specific databases only	Native backup/restore or log shipping	MGN copies everything on the selected disks
Move to Amazon RDS for SQL Server	Native backup/restore (with log restore) or DMS	RDS is database-level only
Very low change rate, tiny downtime, same version	MGN (or log shipping)	Both work; MGN needs less DBA effort
Low bandwidth link, large database	MGN with offline seeding, or backups on AWS Snowball plus log shipping	Initial copy time dominates
Shared-disk clusters / complex HA	Rebuild HA on AWS and move databases with native tools	MGN does not recreate cluster shared storage

Good practice

A good default for a typical estate: **MGN for the first move** (low risk, no DBA rework), then convert databases that are worth modernising to RDS using native backup/restore or DMS, one application at a time.

2. MGN Architecture and How It Works

Understanding the moving parts helps you size the network, open the right ports and troubleshoot calmly during a cutover.

2.1 Components

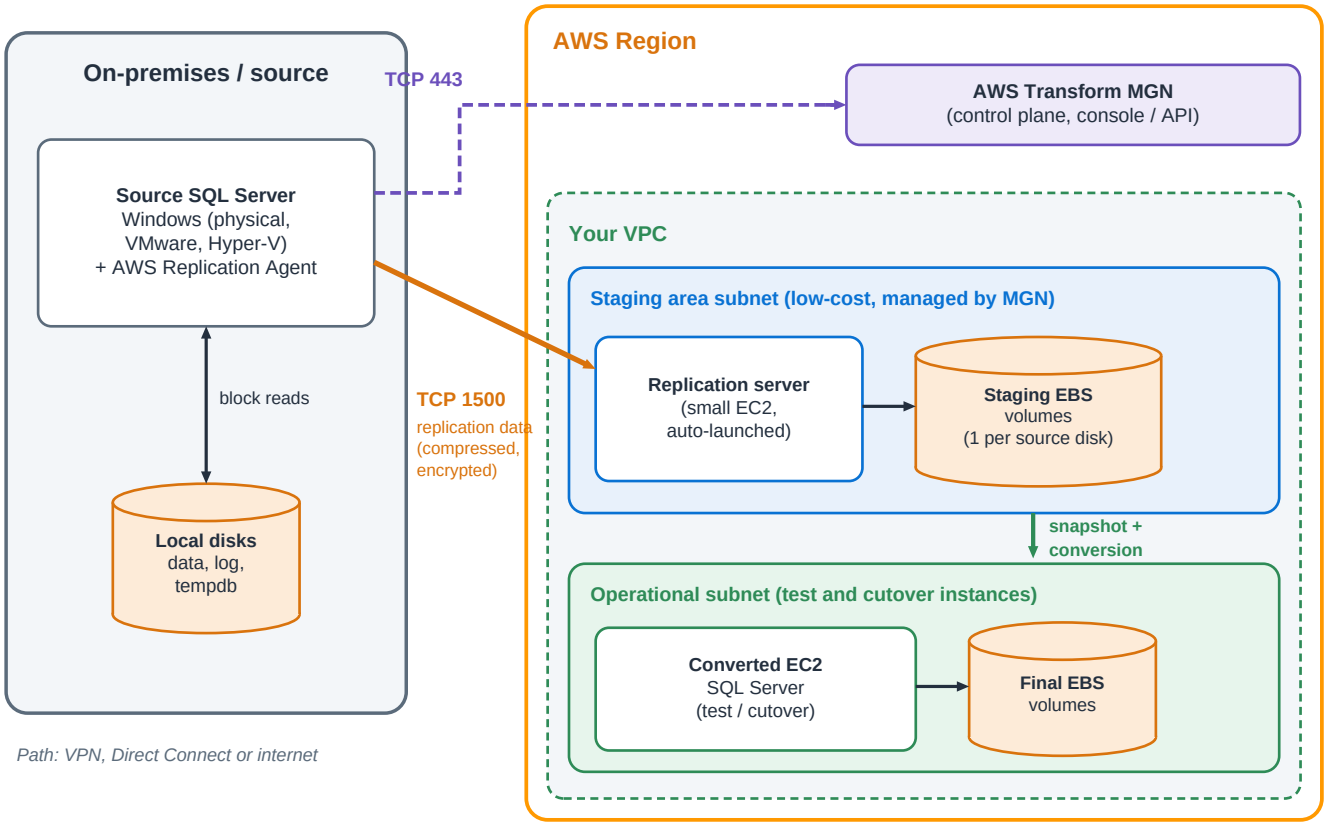


Figure 2.1 - MGN architecture: agent, staging area and test/cutover instances

Component	Where	Role
AWS Replication Agent	On each source server	Reads all disk blocks, then captures every new write, compresses, encrypts and sends them. Installs without a reboot.
MGN service (control plane)	AWS	Tracks servers, lifecycle and replication state; drives test and cutover jobs; reachable over TCP 443.
Replication server	Staging subnet (EC2)	Receives data over TCP 1500 and writes to staging EBS volumes. Launched and scaled automatically (small instance by default).
Staging EBS volumes and snapshots	Staging subnet	One low-cost volume per replicated source disk. MGN takes snapshots to create test and cutover instances.
Conversion server	Staging subnet (temporary)	Adapts the replicated disks (drivers, boot configuration) and creates the target volumes.
EC2 launch template	Per source server	Instance type, subnet, security groups, IAM profile, licensing and tags used for test and cutover.
Post-launch actions (SSM)	Target instance	Runs predefined or custom Systems Manager actions after launch.

2.2 Network ports and endpoints

From	To	Port	Purpose
Source server (agent)	MGN API endpoint (mgn.<region>.amazonaws.com)	TCP 443	Registration, configuration, monitoring. Also S3 to download agent components.
Source server (agent)	Replication server (private or public IP)	TCP 1500	Replication data. Fixed port; a web proxy cannot be used for it.
Replication server	MGN, EC2 and S3 API endpoints	TCP 443	Control, EBS snapshot calls, software downloads. VPC interface endpoints can keep this private.
Test/cutover instance	Domain controllers, DNS, app servers	As needed	Active Directory, name resolution and application traffic.

- Use **Direct Connect or Site-to-Site VPN** with the replication setting *use private IP for data replication* when you do not want replication traffic on the public internet.
- Create **VPC interface endpoints** for MGN, EC2 and S3 if the staging subnet has no internet route.
- Open **outbound TCP 1500** from the source to the staging subnet and **inbound TCP 1500** on the replication server security group (MGN creates the security group; you control the source firewall).
- To **pause** replication, block TCP 1500 or use the console pause action.

2.3 Source-server lifecycle

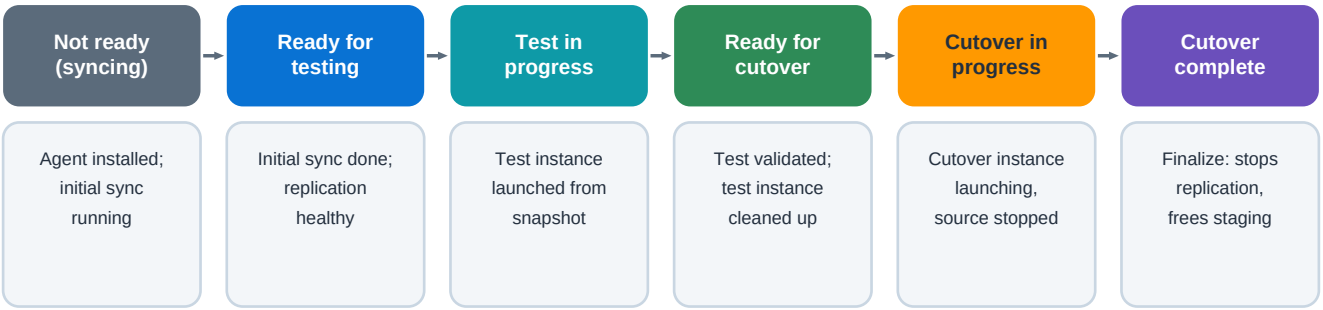


Figure 2.2 - MGN migration lifecycle states for a source server

A server can go through **Test in progress** and back to **Ready for testing** many times. Only **cutover** moves it toward completion. Always **finalize cutover** when you are done so replication stops and staging resources are deleted (this stops charges).

2.4 Data replication states

State	Meaning	What to do
Initial sync	First full copy of all disks is running	Wait. Time depends on data size and bandwidth.
Healthy	Replication is up to date	Normal state; ready to test or cut over.
Backlog / Lagging	Source is writing faster than data is being sent, or the link is slow	Usually clears. If it stays, check bandwidth and the replication server size.
Rescan	Agent is re-reading all blocks and sending the differences	Happens after hard reboots, crashes or disk changes. Faster than initial sync.
Stalled	Replication stopped because of an error	Check alerts: port 1500, credentials, agent service, replication server health.
Paused	You stopped replication on purpose	Resume when ready; MGN resyncs changes.
Disconnected	Agent cannot reach MGN	Check agent service, TCP 443 and the source server.

A clean shutdown or restart of a supported source server does not trigger a rescan. A hard reboot, crash, or adding/removing disks can.

2.5 Quotas to know

Quota (typical defaults)	Value	Note
Active (replicating) source servers per Region	150	Increase through Service Quotas
Non-archived source servers per Region	4,000	Increase through Service Quotas
Source servers in a single job (test/cutover)	200	Plan waves accordingly
Concurrent jobs per Region	20	Applies to launches and cleanups
Post-launch actions per source server	100	Per server or account template

Quotas change. Check the AWS General Reference and Service Quotas for current values before planning waves.

2.6 Cost model

- The **service** is billed per replicating server under AWS pricing terms (check the AWS Transform MGN pricing page for current free period and rates).
- You pay for **staging resources**: replication servers, staging EBS volumes and snapshots, plus data transfer and any VPN/Direct Connect.
- You pay for **test and cutover EC2 instances and their EBS volumes** while they run. Terminate test instances when done.
- Replication cost grows with the number and size of disks and how long you keep replicating before cutover, so avoid starting too early.

3. Planning a SQL Server Migration with MGN

Most MGN issues come from skipped planning: too little bandwidth, missing paths for excluded disks, Active Directory connectivity or licensing surprises. This chapter lists what to decide before installing the agent.

3.1 Assessment checklist

Area	Questions to answer	How to find out
Servers and OS	Windows version and edition; supported by MGN? physical or VM?	Inventory; MGN supported operating systems list
SQL Server	Version, edition, instances, number of databases, clustering/AG	SERVERPROPERTY, SSMS, SQL inventory tools
Disks	Volume layout: OS, data, log, tempdb, backup; sizes; free space (agent needs about 2 GB free on the root volume)	Disk Management, sys.master_files
Change rate	How many GB are written per day and at peak (MB/s)?	Perfmon 'Disk Write Bytes/sec', backup sizes, log generation
Network	Link type and spare bandwidth to AWS; latency; firewall rules	Network team; bandwidth tests
Identity	Domain membership; service accounts; Kerberos SPNs; DNS	AD team; setspn -L
Dependencies	Applications, ETL, linked servers, jobs referencing server names or IPs	Application owners; sys.servers
Licensing	Windows and SQL Server licence model (BYOL or licence-included)	Licence records
Downtime window	How long can the application be down at cutover?	Business owners
Compliance	Encryption, data residency, audit requirements	Security team

```
-- Inventory the files that sit on the disks MGN will replicate
SELECT DB_NAME(database_id) AS db, type_desc, physical_name,
       size * 8 / 1024 AS size_mb
FROM sys.master_files ORDER BY physical_name;

-- Instance and edition facts
SELECT SERVERPROPERTY('MachineName') AS machine, SERVERPROPERTY('Edition') AS edition,
       SERVERPROPERTY('ProductVersion') AS version, SERVERPROPERTY('IsClustered') AS clustered,
       SERVERPROPERTY('IsHadrEnabled') AS ag_enabled;
```

3.2 Network and bandwidth

Two numbers decide whether MGN works smoothly: **time to complete the initial sync** and whether the **steady-state change rate stays below your available bandwidth**. If the source writes more than the link can carry, the server stays in 'lagging' and cutover will be slow.

Data to replicate	100 Mbps	1 Gbps	10 Gbps
500 GB	about 11 hours	about 1.1 hours	about 7 minutes
2 TB	about 1.9 days	about 4.4 hours	about 27 minutes
10 TB	about 9.3 days	about 22 hours	about 2.2 hours
50 TB	about 46 days	about 4.6 days	about 11 hours

Ideal times at 100% link utilisation. Plan for 60-80% efficiency and remember that replication shares the link with production traffic.

- **Throttle** replication in the replication settings if production traffic is sensitive, and schedule heavy initial syncs outside peak hours.
- Use a **dedicated replication server** for very write-heavy or very large servers so they do not slow other servers' replication.
- Prefer **Direct Connect** for multi-terabyte waves. A Site-to-Site VPN is fine for small estates but has per-tunnel limits.
- For very large databases on poor links, consider **seeding with Snowball** or splitting the work: restore backups to EC2 once and then keep it in sync with log shipping.

3.3 AWS landing zone

- **VPC and subnets:** a private **staging subnet** (with a route to MGN endpoints or VPC endpoints) and one or more private **operational subnets** for test and cutover instances, in the AZ you want.
- **Security groups:** an application-tier group for the migrated SQL Server (1433 from app servers, admin access from jump hosts) and the replication security group managed by MGN.
- **IAM:** initialise MGN once per Region (it creates the required service roles). Create credentials for installing the agent with the least-privilege AWS-managed policy for agent installation. Do not use administrator keys on source servers.
- **Encryption:** choose a KMS key for staging and target EBS volumes.
- **Naming and tags:** decide tag conventions (Application, Wave, Owner) to apply from launch templates.

3.4 Licensing decisions

A block-level copy brings your existing licence situation with it. Decide before launch:

Option	How it works	Notes
Windows: licence included (LI)	Target launches with an AWS-provided Windows licence	Pay per hour; the common choice
Windows: BYOL	Use eligible existing licences (Dedicated Hosts or other eligible options)	Check Microsoft licence mobility rules
SQL Server: BYOL (default)	Migrated SQL Server appears as BYOL	You remain responsible for licence compliance
SQL Server: convert to AWS licence-included	Post-launch action converts BYOL to licence-included using AWS License Manager	Supported for Enterprise, Standard and Web editions of SQL Server 2012 to 2022; Windows BYOL with SQL licence-included is not supported

3.5 Active Directory, DNS and names

- A migrated server keeps its **computer name, SID and domain membership**. The VPC must reach a **domain controller** (extend AD over VPN/Direct Connect, or deploy domain controllers or AWS Managed Microsoft AD in AWS).
- If a test instance joins the same domain with the same name as production, conflicts can occur. Test in an **isolated subnet** or with controlled DNS, and never allow two live copies to run against production data.
- The **IP address changes**. Update DNS records, connection strings, linked servers and firewall rules that use IPs. Use DNS names for SQL Server where possible.
- Check **time synchronisation** (Amazon Time Sync Service or domain time) and **SPNs** for Kerberos authentication.

3.6 SQL Server specific preparation

Topic	Why it matters	Action
Disks you exclude from replication	If SQL Server expects a path (for example tempdb on T:\) that does not exist on the target, the instance will not start	Replicate the disk, or create a volume and folders with the same drive letters and permissions before first start
tempdb on local or SSD disk	Cheap to rebuild, but the folder must exist	Replicate it or pre-create the path; later move tempdb to instance store NVMe
Backup disks	Replicating huge backup volumes wastes bandwidth	Exclude backup volumes if they can be rebuilt; keep the backup path valid
Server name inside SQL Server	@@SERVERNAME can differ from the host name if you rename	Keep the same host name, or fix it after launch (section 4.10)
Failover cluster / Always On	Cluster configuration and shared storage are not recreated	Plan to rebuild HA on AWS; migrate nodes as standalone or use native tools
Antivirus, backup agents, hypervisor tools	Can interfere or fail on EC2	Remove or re-configure after launch; exclude MGN agent paths from AV
Disk encryption (BitLocker) and special drivers	Can block conversion or boot	Check MGN prerequisites; test early
Services and jobs referencing IPs or UNC paths	These will not resolve the same way in AWS	Inventory and update after cutover

3.7 Sizing the EC2 target

- Start from the source's real usage (CPU peaks, memory, IOPS, throughput), not from provisioned size. MGN launch settings can propose an instance type from source specifications ('right sizing'); review it.
- Use **memory-optimised instances** for SQL Server buffer pool needs. Check each family's EBS bandwidth and IOPS limits, which cap storage performance.
- Choose EBS types per disk after the move: gp3 for most data/log volumes, io2 Block Express for latency-critical volumes. MGN creates volumes of the type you choose in the launch settings.
- Keep headroom. You can resize after cutover; you should not run at 95% utilisation on day one.

4. Hands-On: On-Premises SQL Server to EC2 with MGN

This chapter walks through a complete migration of a Windows SQL Server from on-premises to Amazon EC2 using MGN. Console steps are described first; CLI commands are shown where they help automation.

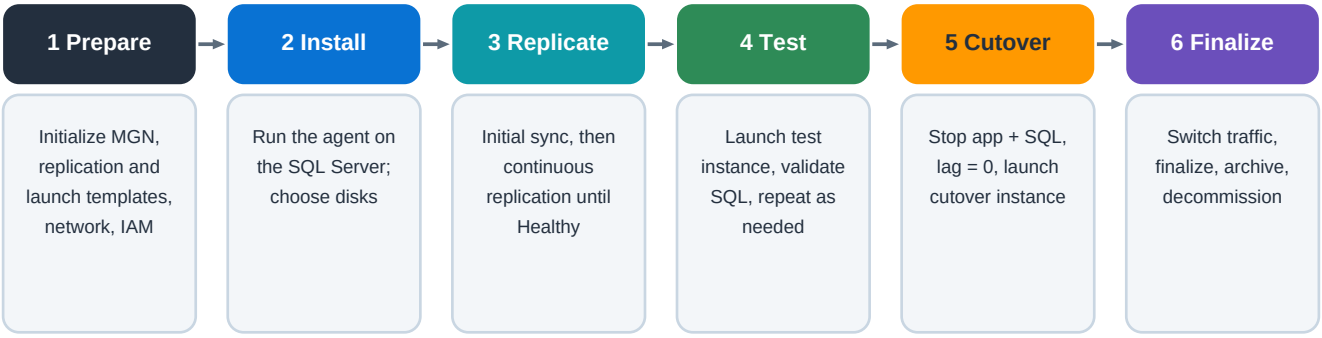


Figure 4.1 - The six stages of an MGN migration

4.1 Step 1: Prepare AWS

Initialize the service

Open the MGN console in the target Region and choose **Get started**. MGN creates the service roles it needs and opens the **replication settings template**. You can also initialise from the CLI.

```
aws mgn initialize-service --region ap-south-1
```

Replication settings template

This template is applied to every new source server. Settings you will normally review:

Setting	Recommendation for SQL Server
Staging area subnet	A private subnet with a route to MGN, EC2 and S3 endpoints (NAT or VPC endpoints) and to your VPN/Direct Connect
Replication server instance type	Start with the default small type; use a larger type for large or write-heavy servers
Dedicated replication server	Enable for heavy-write database servers so they do not share a replication server
Staging disk type	Let MGN choose a low-cost type, or force gp3 for faster initial sync on large servers
EBS encryption	Use a customer-managed KMS key
Data routing	Use private IP for replication over Direct Connect/VPN; keep traffic off the public internet
Bandwidth throttling	Set a limit (Mbps per server) if the link is shared with production
Security group	Let MGN create the replication security group; restrict the source CIDR where possible
Tags	Add Application, Wave and Owner tags

Launch settings and post-launch template

The **launch template** decides how test and cutover instances are created. Define sensible account-level defaults (subnet, security group, IAM instance profile) and override per server for the SQL Server. In **post-launch settings**, allow MGN to install the **SSM agent** (required for other actions) and enable what you need, such as the **CloudWatch agent** and the **SQL Server licence conversion** action.

4.2 Step 2: Create credentials for the agent

- Create an IAM user (or use the console's credential generation) with the AWS-managed **least-privilege policy for installing the MGN agent**.
- Store keys in a secrets vault, use them only for installation, then **deactivate or delete** them after the wave completes.
- Do not use administrator credentials on source servers.

4.3 Step 3: Install the AWS Replication Agent

Verify connectivity, then install. On a Windows SQL Server, open PowerShell as Administrator:

```
# 1) Check the source can reach AWS (control plane)
Test-NetConnection mgn.ap-south-1.amazonaws.com -Port 443

# 2) Download the installer for the target Region (and verify the published SHA-512 hash)
Invoke-WebRequest -Uri "https://aws-application-migration-service-ap-south-1.s3.ap-south-1.amazonaws.com/latest/windows/A
  -OutFile .\AwsReplicationWindowsInstaller.exe

# 3) Install. Without --no-prompt the installer lists volumes and asks which to replicate
.\AwsReplicationWindowsInstaller.exe --region ap-south-1 `
  --aws-access-key-id <ACCESS_KEY_ID> --aws-secret-access-key <SECRET_ACCESS_KEY>
```

On Linux (SQL Server on Linux), download and run the installer script as root or with sudo, for example:

```
sudo python3 aws-replication-installer-init.py --region ap-south-1 \
  --aws-access-key-id <ACCESS_KEY_ID> --aws-secret-access-key <SECRET_ACCESS_KEY>
```

- The installer needs about **2 GB free** on the system drive, does not need a reboot, and prints a **source server ID** (starts with `s-`). Keep it.
- **Disk selection:** select every disk holding data files, log files, tempdb and the OS. Excluding a disk is fine only if you recreate its path on the target (section 3.6).
- On the first install, MGN launches the **replication server** and then opens the data path on **TCP 1500**. If the agent waits at 'connecting', check the firewall and routes.
- Use your standard software deployment tool to install at scale. The installer supports unattended options; see `--help` for your version.

> **Good practice**
> Installing the agent on a busy production SQL Server adds a disk read load during the initial sync. Do it in a quiet period, and use bandwidth throttling if the link is shared. After the sync the overhead is small and mainly proportional to write rate.

4.4 Step 4: Monitor replication

In the MGN console, **Source servers** shows the lifecycle state, replication state, lag and alerts for each server. Wait for **Ready for testing** and **Healthy**. You can also query from the CLI:

```
aws mgn describe-source-servers --region ap-south-1 --filters isArchived=false \
  --query "items[].{id:sourceServerID,host:sourceProperties.identificationHints.hostname,
                   state:lifeCycle.state,repl:dataReplicationInfo.dataReplicationState,
                   lag:dataReplicationInfo.lagDuration}" --output table
```

While waiting, use the time to rehearse DNS changes, prepare the target subnet, configure launch settings and write the cutover runbook.

4.5 Step 5: Configure launch settings for the SQL Server

Setting	What to choose	Notes
Instance type	From sizing (section 3.7) or the right-sizing suggestion	Memory-optimised for SQL Server; mind EBS bandwidth
Subnet and Availability Zone	Private operational subnet in the chosen AZ	Remember where EBS volumes will live
Security groups	App-tier group allowing 1433 from app servers; admin access from jump hosts	Do not reuse the replication security group
IAM instance profile	Role with SSM and CloudWatch permissions (and S3 for backups)	Needed for post-launch actions
Operating system licensing	Licence included, or BYOL as per your agreement	Windows BYOL + SQL licence-included is unsupported
EBS volume types and sizes	gp3 for most; io2 Block Express for latency-critical volumes	Can also be changed after launch
Private IP	Let AWS assign, or copy the source IP only if the CIDR is free and routing is planned	Copying IPs can cause conflicts during testing
Tags	Application, Wave, Owner, Environment	Use 'transfer server tags' where useful
Post-launch actions	SSM agent (default), CloudWatch agent, SQL licence conversion, custom scripts	Custom SSM documents can verify services or update DNS

4.6 Step 6: Launch a test instance and validate

From **Test and cutover > Launch test instances**, or from the CLI:

```
aws mgn start-test --region ap-south-1 --source-server-ids s-0123456789abcdef0
aws mgn describe-jobs --region ap-south-1 --filters sourceServerIDs=s-0123456789abcdef0
```

Isolate the test copy

A test instance is a copy of production with the same host name, domain identity and data. Launch it in a **test subnet** with security groups that block access to production systems (or with controlled DNS) so that jobs, replication and applications do not talk to real production partners. Never let two live SQL Servers write to the same downstream systems.

SQL Server validation checklist

```
-- 1) Are all databases online, and in the expected recovery model?
SELECT name, state_desc, recovery_model_desc, compatibility_level FROM sys.databases;

-- 2) Did crash recovery complete cleanly? (look for 'Recovery completed' for each DB)
EXEC xp_readerrorlog 0, 1, N'Recovery of database';

-- 3) Names and identity
SELECT @@SERVERNAME AS sql_name, SERVERPROPERTY('MachineName') AS host_name;

-- 4) Jobs, linked servers and logins present?
SELECT name, enabled FROM msdb.dbo.sysjobs;
SELECT name, data_source FROM sys.servers;
SELECT COUNT(*) AS logins FROM sys.server_principals WHERE type IN ('S','U','G');

-- 5) Integrity (run on test copy, offloading the source)
DBCC CHECKDB (N'YourDatabase') WITH NO_INFOMSGS, ALL_ERRORMSGS;
```

Validation area	Checks
Instance health	SQL Server and Agent services running; no startup errors; tempdb created; error log reviewed
Data	Database states, row counts on key tables, DBCC CHECKDB, recent data present
Connectivity	App server connects by DNS name; Windows and SQL logins work; Kerberos works if used
Jobs and integrations	Agent jobs enabled (but not yet triggering production targets); linked servers reachable (in test context)
Performance	Representative queries and batch jobs compared with the source baseline
Operations	Backups run; monitoring agents report; patching tool sees the instance

When testing is finished, terminate the test instance (this does not touch replication) and set the server to **Ready for cutover**. In the CLI:

```
aws mgn terminate-target-instances --region ap-south-1 --source-server-ids s-0123456789abcdef0
aws mgn change-server-life-cycle-state --region ap-south-1 --source-server-id s-0123456789abcdef0 \
  --life-cycle state=READY_FOR_CUTOVER
```

4.7 Step 7: Plan the cutover

Item	Detail
Rehearsal	At least one full test with timings: instance launch, SQL start, validation, DNS switch
Downtime window	Agreed with business owners; usually tens of minutes to a couple of hours depending on validation
DNS	Lower TTL on records a day or more ahead; prepare the new record values
Dependencies	App servers, ETL, reporting, linked servers: which move in the same wave?
Communication	Stakeholder notices, support rota, go/no-go criteria, rollback decision time
Rollback plan	Source stays intact and stopped (not decommissioned). If you roll back, point apps to the source and accept loss of writes made on EC2, or reverse-copy them
Approvals	Change record, security review, licence confirmation

4.8 Step 8: Cutover day

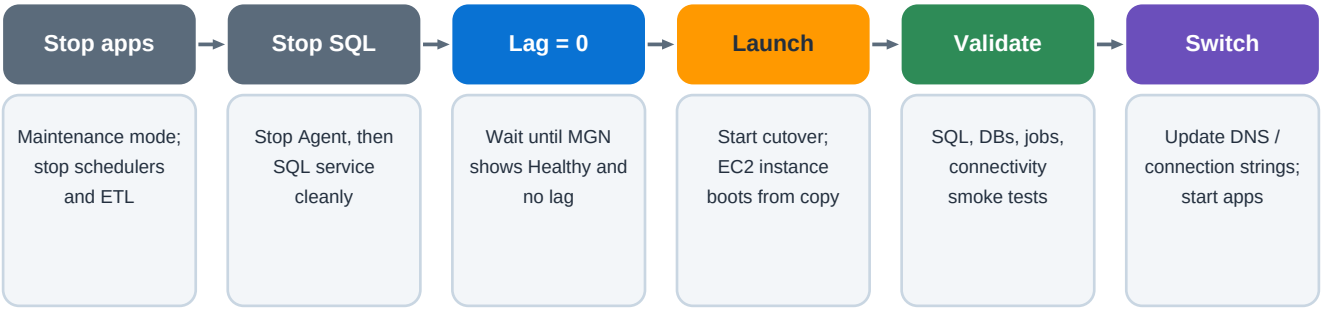


Figure 4.2 - Cutover sequence for SQL Server

Why stop SQL Server cleanly? MGN replicates disk blocks, so pages still in memory (dirty buffers) are not on disk until SQL Server flushes them. A clean shutdown writes everything to disk and closes the log, so the replica starts without crash recovery work and without any chance of lost in-flight data. Replication then only needs to carry the last blocks.

```
# On the SOURCE SQL Server (PowerShell, as administrator)
# 1) Stop application access and SQL Agent jobs first (change window)
Stop-Service SQLSERVERAGENT
# 2) Optional: flush dirty pages before a clean stop
sqlcmd -S localhost -Q "CHECKPOINT"
# 3) Stop the SQL Server service cleanly, then prevent accidental restart
Stop-Service MSSQLSERVER -Force
Set-Service MSSQLSERVER -StartupType Disabled
Set-Service SQLSERVERAGENT -StartupType Disabled
```

In the MGN console, wait until the server shows **Healthy** and the lag is near zero (all the changes from the clean shutdown have arrived), then start the cutover:

```
aws mgn start-cutover --region ap-south-1 --source-server-ids s-0123456789abcdef0
aws mgn describe-jobs --region ap-south-1 --filters sourceServerIDs=s-0123456789abcdef0
```

- When the cutover instance is running, connect through a jump host and perform the **validation checklist** (section 4.6) with SQL Server services started.
- Update **DNS and connection strings**, start application services, and run business smoke tests.
- Hold a **go/no-go** at a pre-agreed time. If it is a no-go, roll back by re-enabling the source SQL Server (set services back to automatic) and pointing applications back.
- After success, run **finalize cutover**. This stops replication and deletes staging resources (replication server, staging volumes, snapshots), which stops their cost.

```
aws mgn finalize-cutover --region ap-south-1 --source-server-id s-0123456789abcdef0
# Later, when the source is decommissioned:
aws mgn mark-as-archived --region ap-south-1 --source-server-id s-0123456789abcdef0
```

4.9 Step 9: Immediately after launch (SQL Server fixes)

Task	Detail
Services	Confirm SQL Server, SQL Agent and related services are running; re-enable services you disabled only on the source
Server name	If the host name changed, fix @@SERVERNAME (below), then restart SQL Server
IP and network settings	Verify the NIC uses the VPC address (DHCP) and DNS servers resolve your domain
Domain	Check secure channel to AD; repair if needed (Test-ComputerSecureChannel -Repair)
Windows activation	Licence-included instances activate through AWS; BYOL needs your activation method
Jobs, alerts, linked servers	Enable jobs; update linked servers, UNC paths and file paths that referenced old IPs/servers
Backups	Take a full backup and verify the first scheduled backup job; update backup targets (for example to S3 or FSx)
Remove old software	Hypervisor tools, on-premises backup/monitoring agents; install CloudWatch and SSM agents if not done by post-launch actions

```
-- Only needed if the Windows host name is different from the old SQL Server name
SELECT @@SERVERNAME;
EXEC sp_dropserver 'OLD-SQL-NAME';
EXEC sp_addserver 'NEW-HOST-NAME', 'local';
-- restart the SQL Server service, then verify
SELECT @@SERVERNAME;
```

5. After the Move: Optimising SQL Server on EC2

MGN gives you the same server, now running on EC2. The next step is making it cloud-appropriate: right-sized, resilient, backed up, patched and monitored.

5.1 First-week checklist

Done?	Task
[ ]	Finalize cutover in MGN; confirm replication and staging resources are gone
[ ]	Review CPU, memory and I/O in CloudWatch; right-size the instance and EBS volumes
[ ]	Set up backups: AWS Backup (VSS-aware snapshots) and/or native backups to S3; test a restore
[ ]	Install and configure the CloudWatch agent; add alarms for CPU, disk, memory, SQL counters
[ ]	Join the instance to AWS Systems Manager; schedule patching with Patch Manager
[ ]	Review security groups, IAM instance profile and EBS encryption; remove unused access
[ ]	Delete the agent installation credentials; uninstall the replication agent from decommissioned sources
[ ]	Decide HA/DR (section 5.3) and licence conversion (SQL licence-included if desired)
[ ]	Keep the source server stopped and intact for the agreed rollback period, then retire it

5.2 Storage and performance tuning

- **Separate volumes** for data, log, tempdb and backups, as on a well-built on-premises server. This is already the case if the source followed that layout.
- **EBS types:** gp3 lets you set IOPS and throughput independently of size; io2 Block Express suits the most latency-sensitive log and data volumes. Modify volume type and size online.
- **tempdb on instance store:** on instance families with local NVMe storage, move tempdb there for very low latency. Recreate the folder at boot with a startup task because instance store does not persist.
- **Instance limits:** confirm the instance's EBS bandwidth and IOPS cap are not lower than your volumes; this is the most common hidden bottleneck.
- **Compare to the baseline:** use the source server's Perfmon history and Query Store to find regressions early.

5.3 High availability and disaster recovery on EC2

Need	Option	Notes
Survive instance or AZ failure	Always On availability group across AZs (WSFC, multi-subnet listener, witness)	You build and operate it; test failover
Survive a Region failure (server level)	AWS Elastic Disaster Recovery (block-level replication to another Region)	Can be enabled as an MGN post-launch action; recovery is crash-consistent
Survive a Region failure (database level)	Distributed AG, log shipping, or backups replicated to another Region	Lower RPO and application-consistent
Protect against deletion and corruption	Native full/diff/log backups to S3 and AWS Backup snapshots	Test restores regularly
Least admin effort	Replatform to Amazon RDS Multi-AZ (chapter 6)	Managed HA and backups

5.4 Clean up and cost control

- **Finalize cutover** promptly: staging servers, volumes and snapshots keep costing until you do.
- **Terminate test instances** and delete test AMIs/snapshots created during validation.
- **Archive** source servers in MGN after cutover so they do not count as active.
- Keep the on-premises source for the rollback period only; then decommission and reclaim licences according to your agreements.
- Tag everything and review costs weekly during the first month; right-size instances and volumes based on measurements.

5.5 Troubleshooting MGN for SQL Server

Symptom	Likely cause	What to do
Agent installs but never reaches 'Initial sync'	TCP 1500 blocked, or replication server unreachable	Test-NetConnection to the replication server on 1500; check route, NACL, firewall, security group
Server shows Stalled	Link down, agent service stopped, credential or proxy problem	Read the alert; restart the agent service; confirm 443 and 1500; note a web proxy cannot carry 1500
Constant Lagging / Backlog	Change rate exceeds bandwidth, or replication server too small	Add bandwidth, schedule heavy jobs outside the window, use a larger or dedicated replication server
Rescan after reboot	Hard reboot, crash or disk change	Avoid hard reboots; clean shutdowns do not rescan; wait for it to finish
Test instance does not boot or has no network	Driver or boot issue, wrong subnet or security group	Check the launch history and EC2 system log/screenshot; verify the launch template and subnet routing
SQL Server fails to start on the test instance	Missing path for an excluded disk (tempdb, data, log), permissions	Create the missing drive/folders with the same permissions; review the SQL error log
Databases show 'Recovery pending' or long recovery	Crash-consistent copy of a busy server	Wait for recovery; for production, stop SQL cleanly before cutover next time
Cannot log in with domain accounts	Secure channel broken or no route to a domain controller	Fix routing/DNS; Test-ComputerSecureChannel -Repair; check time sync
Application cannot find the server	IP changed, DNS not updated	Update DNS and connection strings; flush client caches
Linked servers or jobs fail	Hard-coded IPs, UNC paths, firewall rules	Update objects and add the new source IP to partner firewalls
Licence or activation warnings	BYOL activation not configured, or mixed licensing	Apply licence settings; use the MGN licence conversion action where supported
Test instance interferes with production	Test copy reached production systems	Isolate test subnets; never test with live write paths

6. Second Leg: SQL Server on EC2 to Amazon RDS

After MGN has rehosted the server to EC2, you can modernise selected databases to Amazon RDS for SQL Server. Because RDS is a managed database service with no operating system, MGN cannot do this. You move databases instead of servers, using SQL Server's own tools or AWS DMS.

6.1 Why MGN is not used for EC2 to RDS

- MGN replicates **disks of a server** and launches an EC2 instance. RDS has no disks or OS you can attach.
- RDS needs **logical, database-level** movement: backup/restore, log shipping, replication or DMS.
- RDS has restrictions (no sysadmin, no OS access, no restoring system databases), so each database must be assessed and the server-level objects re-created.

The good news is that the EC2 server migrated by MGN is a normal SQL Server in your VPC. It sits close to RDS, with fast private networking, so database moves are quick and easy to rehearse.

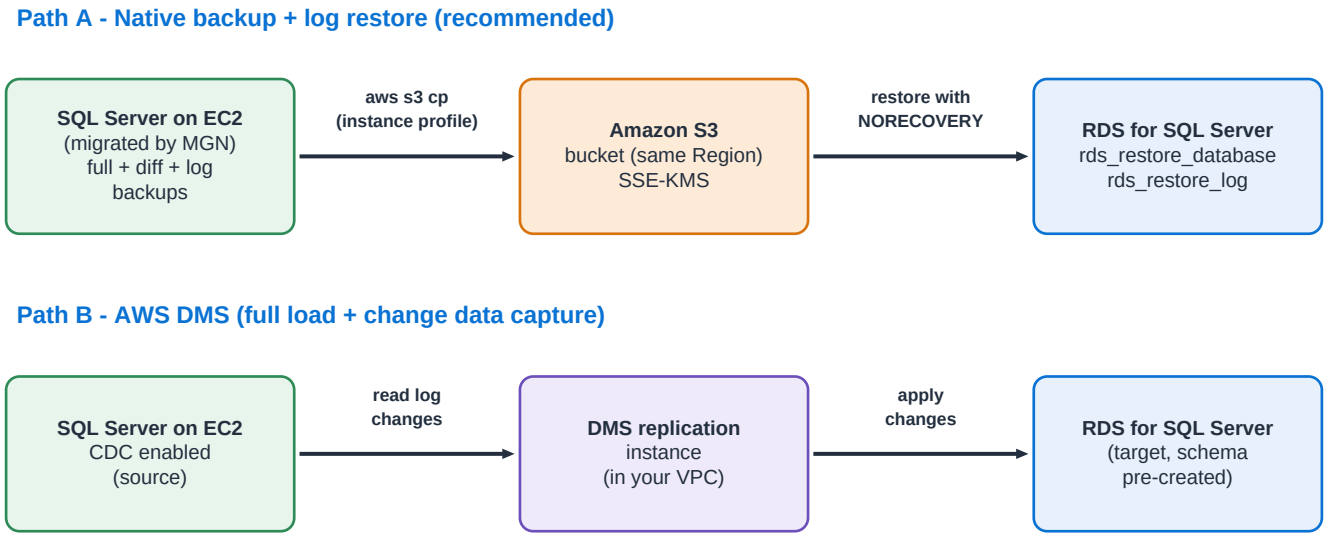


Figure 6.1 - Two ways to move databases from EC2 to RDS

6.2 Assess RDS readiness

Check	Why it matters	How to check
SQL Server version of the source	RDS target version must be the same or higher; you cannot restore a newer backup on an older engine	SERVERPROPERTY('ProductVersion')
Enterprise-only features in use	A backup that uses them cannot be restored to RDS Standard	sys.dm_db_persisted_sku_features
Database size and count	Storage, instance class and the per-instance database limit	sys.master_files; RDS limits documentation
Collation and time zone	Fixed when the RDS instance is created	SERVERPROPERTY('Collation'); Windows time zone
SQL Agent jobs and steps	CmdExec/PowerShell steps and local file access do not run on RDS	msdb.dbo.sysjobsteps (subsystem)
CLR, FILESTREAM, PolyBase, ML Services, Stretch DB	Unsupported or restricted on RDS	sys.assemblies, sys.master_files
Logins, linked servers, Database Mail	Must be re-created; SIDs should match	sys.server_principals, sys.servers
SSIS, SSRS, SSAS on the same server	Not part of a database restore	Inventory; option-based support or keep on EC2
Cross-database and cross-server dependencies	Databases that depend on each other should move together	Dependency analysis
Performance needs	IOPS and throughput caps of the instance class and storage	CloudWatch and Perfmon from the EC2 server

6.3 Choosing the method

Method	Downtime	Best for	Remarks
Native full backup + restore	Backup + copy + restore time	Small and medium databases	Simplest; no ongoing sync
Native full + differential + log restore	Minutes	Large or busy databases	Physical copy; you script the log copy; RDS has no built-in log shipping
AWS DMS (full load + CDC)	Minutes	Selective tables, continuous sync	Pre-create schema; moves data only; CDC prerequisites
Transactional replication	Minutes	Object-level control	Needs distributor configuration; check RDS support for your version
AWS Migration Hub Orchestrator	Same as native backup	Many databases, repeatable workflow	Automates native backup and restore steps
BACPAC / SqlPackage	Long	Small databases only	Portable but slow

6.4 Prepare the RDS target

- **Network:** create the RDS instance in private subnets (a DB subnet group across two or more AZs) in the same VPC (or peered) as the EC2 SQL Server. Allow TCP 1433 from the EC2 and application security groups.
- **Size:** use CloudWatch and Windows performance data from the EC2 SQL Server (CPU, memory, IOPS, throughput) to pick the instance class and storage type. RDS performance is limited by both the class and the volume.
- **Collation and time zone** must match the source; set them when you create the instance.
- **Option group:** add **SQLSERVER_BACKUP_RESTORE** with an IAM role that can read the migration bucket (and use your KMS key).
- **S3 bucket:** same Region as the RDS instance, encryption enabled, access limited to the EC2 instance role and the RDS role.
- **Start with Single-AZ** for the migration and enable Multi-AZ after cutover (a common approach); set backup retention and Performance Insights.

```
aws rds create-option-group --option-group-name sql-native-backup --engine-name sqlserver-se \
  --major-engine-version 16.00 --option-group-description "Native backup and restore"
aws rds add-option-to-option-group --option-group-name sql-native-backup --apply-immediately \
  --options "OptionName=SQLSERVER_BACKUP_RESTORE,OptionSettings=[{Name=IAM_ROLE_ARN,Value=arn:aws:iam::111122223333:role/
```

6.5 Path A: native backup and log restore

Step 1: Take backups on the EC2 SQL Server and upload to S3

```
-- Full backup (not COPY_ONLY if you will apply differentials later)
BACKUP DATABASE SalesDB TO DISK = 'E:\Backups\SalesDB_full.bak' WITH COMPRESSION, CHECKSUM, STATS = 10;
-- Later: differential and log backups
BACKUP DATABASE SalesDB TO DISK = 'E:\Backups\SalesDB_diff1.bak' WITH DIFFERENTIAL, COMPRESSION, CHECKSUM;
BACKUP LOG SalesDB TO DISK = 'E:\Backups\SalesDB_log_0001.trn' WITH COMPRESSION, CHECKSUM;
```

```
# Uses the EC2 instance profile (no keys needed); the role needs s3:PutObject on the bucket
aws s3 cp E:\Backups\SalesDB_full.bak s3://my-migration-bucket/sales/
aws s3 sync E:\Backups\ s3://my-migration-bucket/sales/ --exclude "*" --include "*.trn" --include "*_diff*.bak"
```

Step 2: Restore on RDS with NORECOVERY and keep it current

```
EXEC msdb.dbo.rds_restore_database @restore_db_name = 'SalesDB',
     @s3_arn_to_restore_from = 'arn:aws:s3:::my-migration-bucket/sales/SalesDB_full.bak',
     @with_norecovery = 1;
EXEC msdb.dbo.rds_task_status @db_name = 'SalesDB';        -- progress

EXEC msdb.dbo.rds_restore_database @restore_db_name = 'SalesDB', @type = 'DIFFERENTIAL',
     @s3_arn_to_restore_from = 'arn:aws:s3:::my-migration-bucket/sales/SalesDB_diff1.bak',
     @with_norecovery = 1;

EXEC msdb.dbo.rds_restore_log @restore_db_name = 'SalesDB',
     @s3_arn_to_restore_from = 'arn:aws:s3:::my-migration-bucket/sales/SalesDB_log_0001.trn',
     @with_norecovery = 1;
```

Step 3: Cut over

- Stop the application (or set the source database `READ_ONLY`), take the **tail log backup**, upload it and restore it with recovery (`@with_norecovery = 0`, or run `EXEC msdb.dbo.rds_finish_restore @db_name = 'SalesDB'` if the database is left restoring).
- Fix orphaned users, re-create jobs and server objects (section 6.7), repoint connection strings to the **RDS endpoint** and start the application.
- Enable Multi-AZ and automated backups, take a manual snapshot, and run validation.

Rules for native restore into RDS

Source version must be the same as or older than the RDS engine version.
Enterprise-only features inside the database prevent restoring to Standard.
Do not take other full backups between the base full and the differential unless they are COPY_ONLY.
If the source uses TDE, move the certificate with the RDS TDE option procedures or remove TDE before backup.
Very large backups should be striped into several files; check current size limits.

6.6 Path B: AWS DMS from EC2 to RDS

Step	Action
1	Create the target schema on RDS (tables, keys, indexes, procedures) from scripts or a schema-only backup/restore of a copy. DMS moves data, not most programmability.
2	Source prerequisites: databases in FULL recovery, change capture enabled (for example `EXEC sys.sp_cdc_enable_db` and tables), a DMS login with the needed rights, enough transaction log retention.
3	Create a DMS replication instance in the VPC with access to both the EC2 SQL Server and RDS (security groups on 1433).
4	Create source and target endpoints and test connections.
5	Create a task: full load + CDC, table mappings, LOB settings, data validation.
6	Monitor latency. At cutover, stop writes, wait for zero latency, validate counts and checksums, then repoint applications.
7	Re-create secondary indexes, constraints, triggers and jobs that DMS did not move; update statistics.

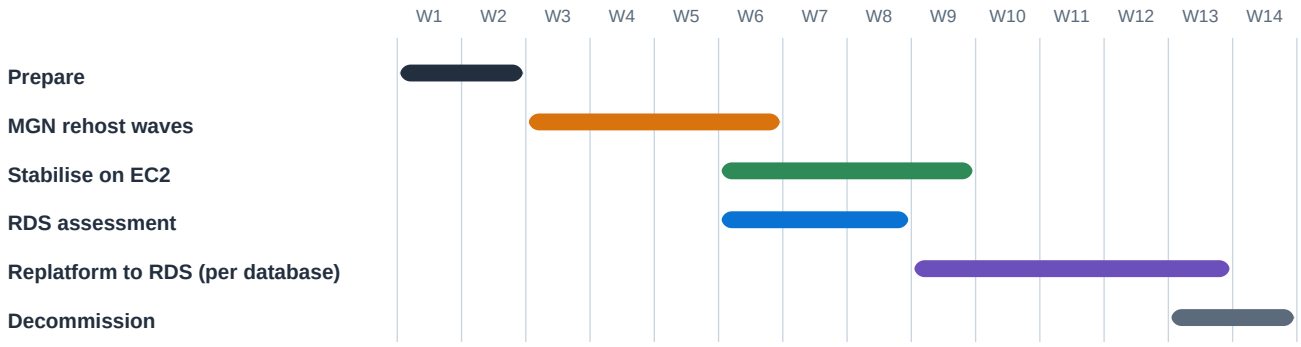
6.7 Server-level objects

Object	How to move it to RDS
Logins and passwords	Script logins with SIDs and password hashes from EC2, create on RDS before or right after restore; fix orphaned users with ALTER USER ... WITH LOGIN
SQL Agent jobs, operators, alerts	Script from SSMS; adjust steps for RDS limits; replace CmdExec/PowerShell steps with AWS services
Linked servers, credentials	Re-create and test from the RDS network location
Database Mail	Re-create profiles and test the SMTP route
Server settings (sp_configure)	Map to DB parameter group settings
SSIS/SSRS/SSAS	Keep on EC2, re-host with RDS options if supported, or redesign

6.8 Validation, cutover and clean-up

When	Action	Owner
T - 1 week	Rehearse the full path with production-size data; record timings	DBA
T - 1 day	Confirm log restore is current; lower DNS TTL; freeze deployments	DBA / App
T - 15 min	Stop application writes	App team
T	Tail log backup, upload, restore with recovery; fix orphaned users	DBA
T + 15 min	Repoint apps to the RDS endpoint; start; smoke tests	App team
T + 2 hours	Go/no-go. Rollback = point back to EC2 SQL Server (kept intact and stopped from writes)	Project lead
T + 1 week	Enable Multi-AZ if deferred; tune; review CloudWatch and Performance Insights	DBA
T + 2-4 weeks	Final backup of the EC2 database, stop the EC2 SQL Server, snapshot and later terminate it	DBA / Finance

6.9 A combined programme plan



Illustrative weeks (W1-W14). Real durations depend on estate size, bandwidth and change windows.

Figure 6.2 - Example programme: MGN rehost first, RDS replatform later

Run the two legs as **separate changes**. After the MGN wave, leave the EC2 SQL Server stable for a few weeks. That gives you a clean performance baseline on AWS and removes the data-centre dependency before you start the more careful database-by-database RDS work. Databases that do not suit RDS can stay on EC2 permanently and be optimised there (chapter 5).

> **When to skip the EC2 stop**
> If you already know a database is RDS-compatible and your downtime window allows, you can migrate **directly from on-premises to RDS** with the same native backup/log restore or DMS method, without MGN. Use MGN when you want to move the whole server unchanged, quickly and with minimal risk.

Appendix A: MGN Command Cheat Sheet

Task	AWS CLI
Initialize MGN in a Region	aws mgn initialize-service --region ap-south-1
List active source servers	aws mgn describe-source-servers --region ap-south-1 --filters isArchived=false
Launch a test instance	aws mgn start-test --region ap-south-1 --source-server-ids s-xxxxxxxx
Check job progress	aws mgn describe-jobs --region ap-south-1
Remove test instances	aws mgn terminate-target-instances --region ap-south-1 --source-server-ids s-xxxxxxxx
Mark ready for cutover	aws mgn change-server-life-cycle-state --source-server-id s-xxxxxxxx --life-cycle state=READY_FOR_CUTOVER
Start cutover	aws mgn start-cutover --region ap-south-1 --source-server-ids s-xxxxxxxx
Finalize cutover	aws mgn finalize-cutover --region ap-south-1 --source-server-id s-xxxxxxxx
Archive a source server	aws mgn mark-as-archived --region ap-south-1 --source-server-id s-xxxxxxxx
Pause / resume replication	aws mgn pause-replication / resume-replication --source-server-id s-xxxxxxxx

Commands and options are shown for orientation. Confirm syntax with `aws mgn help` *for your CLI version.*

Appendix B: Cutover Readiness Checklist

Done?	Item
[ ]	Replication status Healthy; lag near zero in the last 24 hours
[ ]	At least one successful test launch and full validation, including application test
[ ]	Launch settings final: instance type, subnet, security groups, IAM profile, licensing, tags, EBS types
[ ]	Post-launch actions configured and tested (SSM, CloudWatch agent, licence conversion if used)
[ ]	Active Directory and DNS connectivity from the operational subnet confirmed
[ ]	DNS TTL lowered; new records prepared; connection string changes ready
[ ]	Runbook with times, owners, go/no-go and rollback decision point agreed
[ ]	Backups of the source taken and verified; source will remain intact and stopped
[ ]	Monitoring, alerting and support rota ready for the new instance
[ ]	Stakeholders informed; change approved

Appendix C: Glossary

Term	Meaning
AWS Transform MGN / MGN	AWS lift-and-shift replication service (formerly AWS Application Migration Service)
AWS Replication Agent	Software on the source server that replicates disk blocks to AWS
Staging area subnet	Subnet holding replication servers and low-cost staging volumes
Replication server	Small EC2 instance receiving replicated data
Conversion server	Temporary EC2 instance that makes replicated disks bootable on EC2

Term	Meaning
Test / cutover instance	EC2 instance launched from replicated data for validation / final migration
Launch template	EC2 launch settings applied to test and cutover instances
Post-launch action	Automated Systems Manager action run after launch
Rehost / replatform	Move as-is to EC2 / move to a managed service with limited change
Crash-consistent	State equal to a disk after a sudden power loss; SQL Server recovers on startup
CDC	Change Data Capture on SQL Server, used by DMS for ongoing replication
DMS	AWS Database Migration Service
RDS	Amazon Relational Database Service
RPO / RTO	Recovery point / time objective

Appendix D: Where to Learn More

MGN and RDS evolve quickly (MGN was renamed AWS Transform MGN in June 2026). Treat this guide as an architecture and process reference and confirm limits, supported operating systems and command syntax in the current AWS documentation.

Topic	Where to look
MGN user guide	AWS Transform MGN (formerly Application Migration Service) User Guide: quick start, replication settings, launch settings, post-launch actions, FAQ
SQL Server rehost pattern	AWS Prescriptive Guidance: 'Migrate an on-premises Microsoft SQL Server database to Amazon EC2 using Application Migration Service'
Licence conversion	AWS Microsoft Workloads blog: converting SQL Server licences migrated with MGN
SQL Server on EC2	Microsoft SQL Server on Amazon EC2 user guide; AWS Prescriptive Guidance for SQL Server on EC2 HA/DR
Migration methods	AWS Prescriptive Guidance: Migrating Microsoft SQL Server databases to the AWS Cloud
RDS for SQL Server	Amazon RDS User Guide: native backup and restore, Multi-AZ, read replicas, common DBA tasks
DMS	AWS Database Migration Service User Guide: SQL Server as a source and as a target
Quotas and pricing	AWS General Reference (MGN quotas) and the AWS Transform MGN pricing page